MBLP131 · HAFTA 3

İşletim Sistemleri Temelleri

Hafta 3 · İşletim Sistemi Hizmetleri, Sistem Çağrıları ve Süreçler

İşletim sistemi bir program için ne yapar ve çalışan bir program tam olarak nedir?

Elektronik Haberleşme Teknolojisi Programı · Meslek Yüksekokulu · İstanbul Okan Üniversitesi

GEÇEN HAFTA

Dört Fikirde 2. Hafta

İki bakış

Genişletilmiş makine (temiz soyutlamalar) ve kaynak yöneticisi (donanımı paylaştırmak).

İkili mod

İşlemcide bir mod biti: kullanıcı modu kısıtlıdır, çekirdek modunda her şey yapılabilir.

İlk sistem çağrısı

printf() sonunda write(1, …) oldu; strace çekirdeğe giden her isteği gösterdi.

Çağrılar pahalıdır

Test makinemizde bir getpid() sistem çağrısı ≈ 116 ns sürdü; bir fonksiyon çağrısının yaklaşık 76 katı.

Bugün: sistem çağrısı kapısının ardında hangi hizmetler var, bir çağrı çekirdeğe gerçekte nasıl gider ve "süreç" nedir?

Bugünün Gündemi

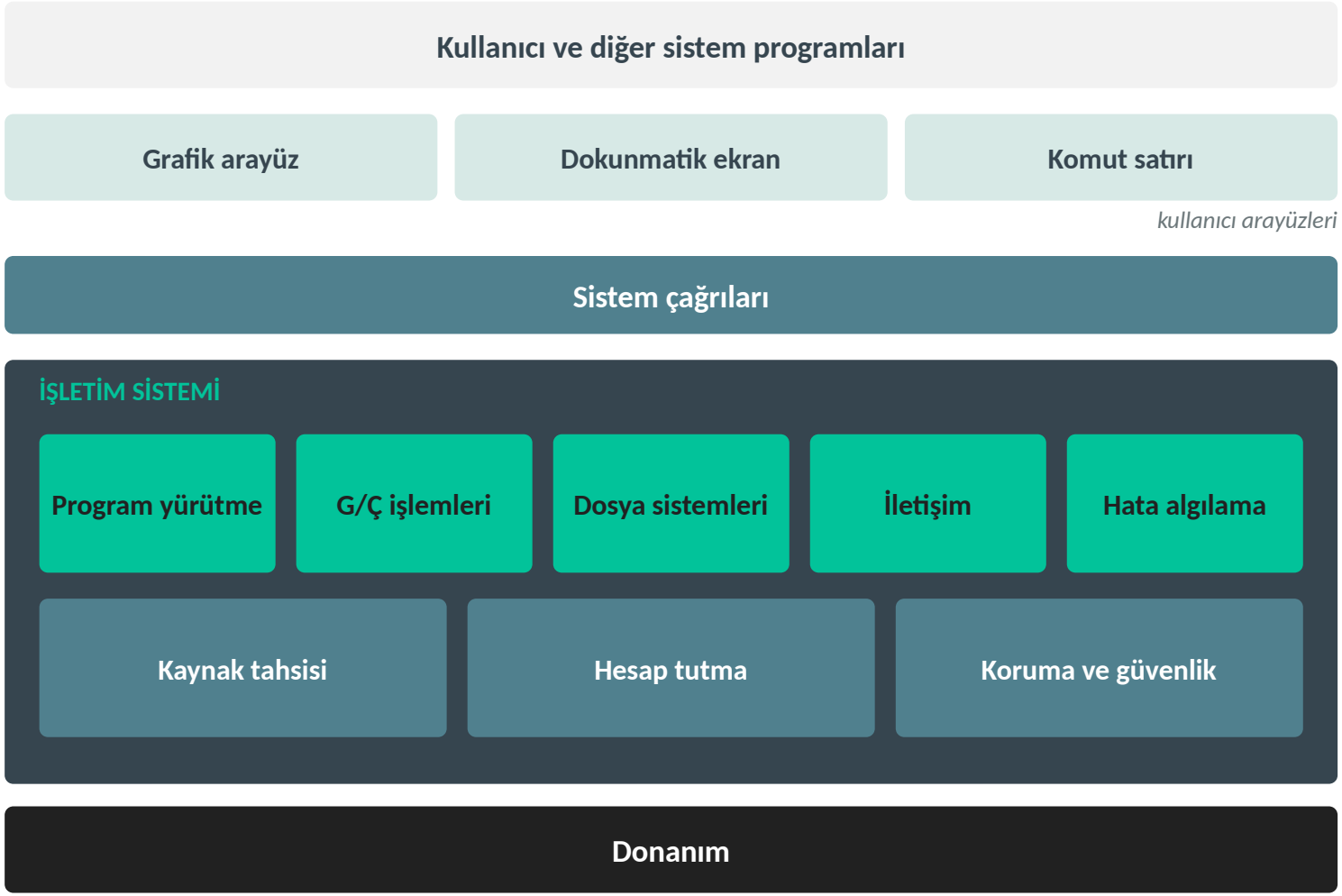
1. **Hizmetler** İşletim sisteminin kullanıcılara, programlara ve kendisine sundukları.
2. **Sistem çağrıları** API ve sistem çağrısı, sistem çağrısı tablosu, tuzak, parametre aktarımı.
3. **Süreçler** Program ve süreç, bellek düzeni, beş durum, süreç denetim bloğu (PCB).
4. **Bağlam değişimi** Ne kaydedilir, neden saf ek yüktür ve maliyeti ne kadardır?
5. **Uygulama** C programları: çıplak sistem çağrısı, bellek haritası, fork / exec / wait, fork ağaçları.

BÖLÜM 1

İşletim Sistemi Hizmetleri

İşletim sisteminin size, programlarınıza ve kendisine sağladıkları.

İşletim Sistemi Hizmetlerine Genel Bakış



İki grup

- **Nane yeşili sıra:** kullanıcının ve programlarının işini görmesine yardım eden hizmetler.
- **Gri sıra:** birçok kullanıcı ve program sistemi paylaşırken sistemin kendisini verimli ve güvenli tutan hizmetler.
- Programlar her hizmete aynı kapıdan ulaşır: sistem çağrıları.

HİZMETLER

Kullanıcıya Yardım Eden Hizmetler

Kullanıcı arayüzü

Komut satırı (bash, PowerShell), grafik masaüstü, dokunmatik ekran ya da komutlardan oluşan bir toplu iş dosyası.

Program yürütme

Bir programı belleğe yüklemek, çalıştırmak ve sonlandırmak: normal biçimde ya da bir hata koduyla.

G/Ç işlemleri (I/O)

Programlar aygıtları asla doğrudan sürmez; G/Ç'yi onlar adına işletim sistemi yapar (2. hafta: ikili mod).

Dosya sistemi işlemleri

Dosya ve dizin oluşturmak, silmek, okumak, yazmak, aramak; izinleri yönetmek.

İletişim

Süreçler arasında veri alışverişi: aynı bilgisayarda (boru, paylaşımlı bellek) ya da ağ üzerinden (soket).

Hata algılama

Hataları fark edip ele almak: bir disk okuması başarısız olur, yazıcının kâğıdı biter, bir program sıfıra böler.

HİZMETLER

Sistemin İyi Çalışmasını Sağlayan Hizmetler

Tek bir makinede çok sayıda kullanıcı ve program varken işletim sisteminin kendisi için de hizmetlere ihtiyacı vardır.

Kaynak tahsisi

İşlemciyi kim alacak, ne kadar bellek, hangi yazıcı? Bunu zamanlayıcılar ve ayırıcılar yapar (4–7. haftalar).

Linux: ulimit -a süreç başına sınırları gösterir.

Hesap tutma ve günlük

Kimin ne kadar işlemci zamanı, bellek ve disk kullandığını izlemek: faturalama, istatistik ve ayar için.

Linux: ps, top, journalctl; Windows: Olay Görüntüleyicisi.

Koruma ve güvenlik

Koruma: süreçler birbirinin belleğine dokunamaz. Güvenlik: kullanıcılar kimlik doğrular; saldırılar kaydedilir.

Linux: dosya izinleri (11. hafta), parolalar.

Zincir en zayıf halkası kadar güçlüdür: tahmin edilen tek bir parola, geçen hafta gördüğümüz bütün koruma donanımını boşa çıkarır.

Komut Satırı ve Grafik Arayüz

```
# bir komut satırı oturumu (Linux'ta bash)
$ ls -l report.txt      # dosya sistemi hizmeti
$ cp report.txt backup/ # dosya sistemi hizmeti
$ ./myprog &            # program yürütme
$ ps                    # durum bilgisi
$ kill 4242             # süreç denetimi

# her komut, kabuğun sizin için başlattığı
# ayrı bir programdır (3. bölüm: fork/exec)
```

Kabuk (shell) çekirdeğin parçası değildir: bir komut okuyup işletim sisteminden onu çalıştırmasını isteyen sıradan bir programdır.

Hangisi daha iyi?

- **Komut satırı (CLI):** uzmanlar için hızlı, betiklenebilir, yavaş bir ağ bağlantısında da çalışır; komutları hatırlamak gerekir.
- **Grafik arayüz (GUI):** pencereler, simgeler, fare; öğrenmesi kolay, otomatikleştirmesi zor.
- **Dokunmatik ekran:** fare yerine hareketler; telefonlar ve tabletler.
- Sistem yöneticilerinin çoğu ikisini de kullanır. 9. haftadan itibaren Linux kabuğunda çalışacağız.

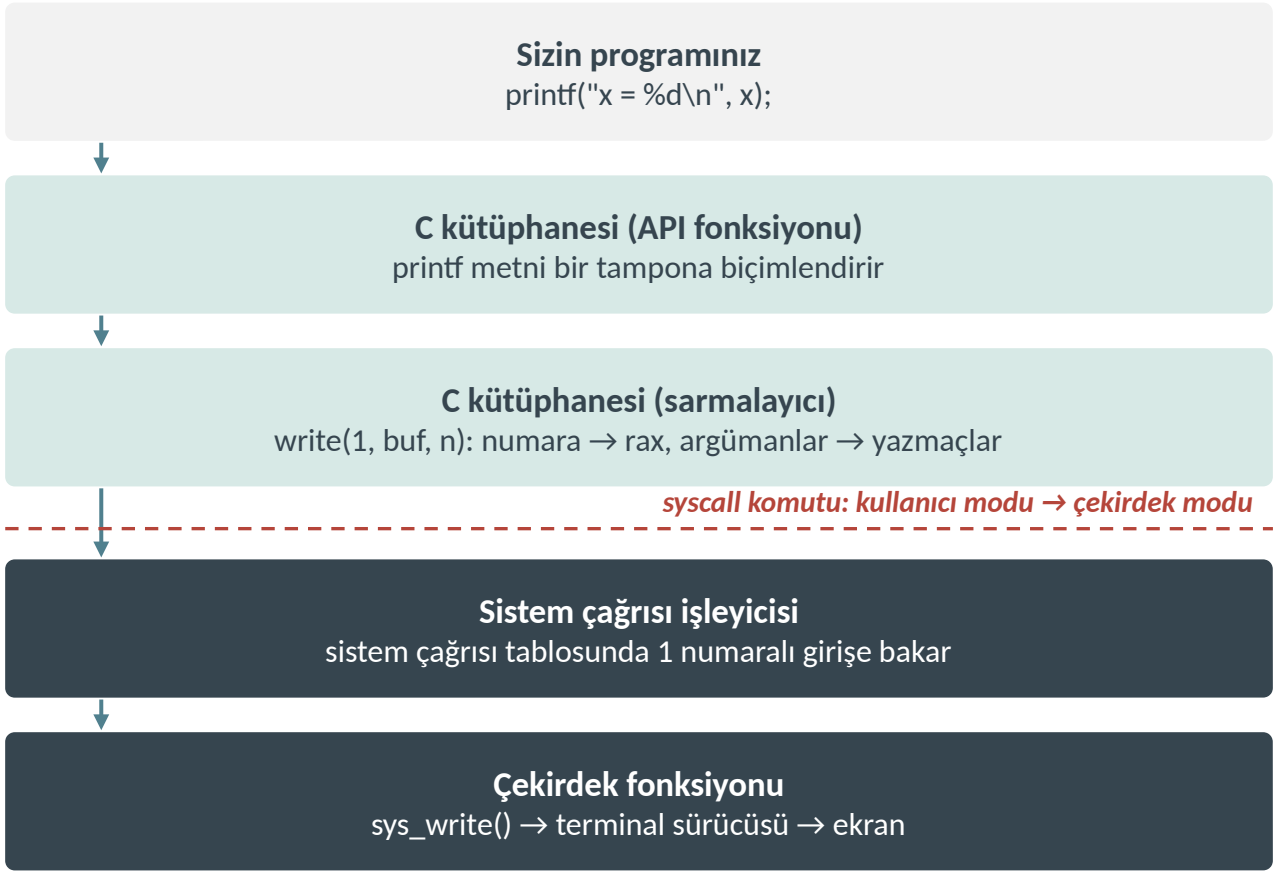
BÖLÜM 2

Ayrıntılı Sistem Çağrıları

Bir kullanıcı programından çekirdeğe açılan tek kapı.

API, Kütüphane Sarmalayıcısı ve Sistem Çağrısı

Programcılar neredeyse hiçbir zaman sistem çağrısını doğrudan yürütmez. printf() ile donanım arasında katmanlar vardır.



Üç sözcük, üç anlam

- **API** (uygulama programlama arayüzü): programcının çağırdığı belgelenmiş fonksiyonlar; ör. POSIX ya da Windows API.
- **Sarmalayıcı (wrapper):** sistem çağrısıyla aynı adı taşıyan minik kütüphane fonksiyonu; yalnızca yazmaçları doldurur ve tuzak kurar.
- **Sistem çağrısı:** çekirdeğe asıl giriş.
- Her API fonksiyonu sistem çağrısı yapmaz: strlen() ve sqrt() hiç yapmaz; printf() yalnızca tamponu boşaltılınca yapar.

Neden Bir API'ye Göre Programlanır?

Yaygın üç API — her biri kendi platformunun gerçek sistem çağrılarını gizler.

POSIX API

- Linux, macOS, BSD'ler.
- C kütüphanesi sağlar (glibc, musl, ...).
- write(), fork(), open()...

Windows API

- Windows.
- CreateProcess(), ReadFile()...
- İçeride ntdll.dll'i çağırır; asıl çekirdek çağrılarını o yapar (NtCreateUserProcess...).

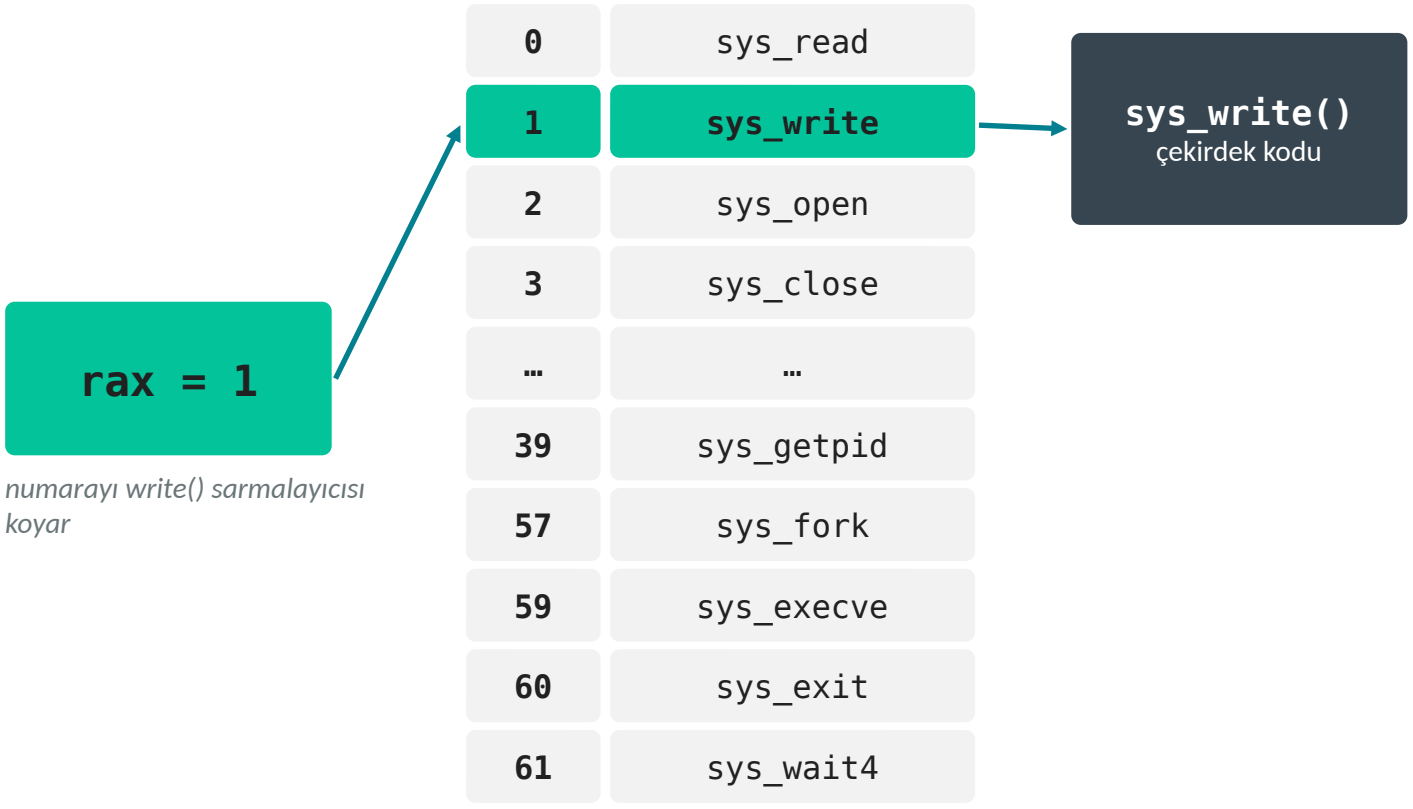
Java API

- Java Sanal Makinesi olan her işletim sistemi.
- Files.readAllBytes(), ProcessBuilder...
- JVM bunları ana işletim sisteminin sistem çağrılarına çevirir.

Taşınabilirlik: write, 64 bit x86 Linux'ta 1, 32 bit x86 Linux'ta 4, ARM64 Linux'ta 64 numaralı sistem çağrısıdır. API adı write() ise her yerde aynıdır.

Her Sistem Çağrısının Bir Numarası Vardır

Çekirdek bir tablo tutar: sistem çağrısı numarası → işi yapan çekirdek fonksiyonunun adresi.



numarayı write() sarmalayıcısı koyar

sistem çağrısı tablosu (x86-64 Linux)

Neden bir tablo?

- Kullanıcı programı çekirdeğe yalnızca tek bir sabit giriş noktasından girebilir; asla rastgele bir çekirdek adresine atlayamaz.
- Numara hizmeti seçer; çekirdek geçerli olup olmadığını denetler, sonra tablodaki fonksiyonu çağırır.
- x86-64 üzerinde Linux 6.8'de 0'dan 461'e numaralanmış yaklaşık 370 sistem çağrısı vardır.

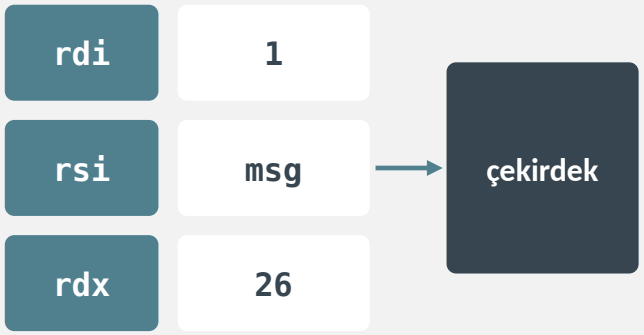
Bir write(): Çekirdeğe Yolculuk, Adım Adım



açık = kullanıcı modu koyu = çekirdek modu

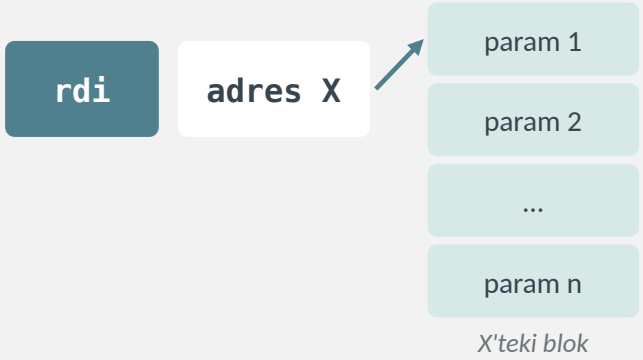
İşletim Sistemine Parametre Aktarmanın Üç Yolu

1. Yazmaçlarda



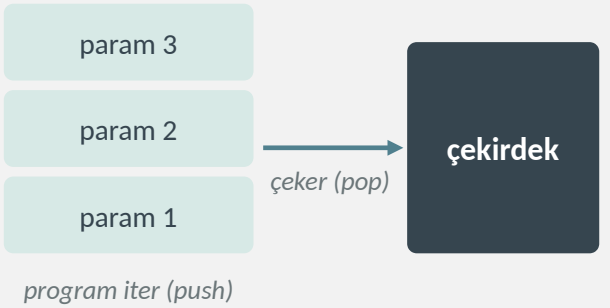
En hızlısı. Yazmaç sayısıyla sınırlıdır (x86-64 Linux'ta 6).

2. Bir bellek bloğunda



İstenen sayıda parametre; yazmaçta yalnızca adres taşınır. Örnek: clone3() bir yapıya işaretçi alır.

3. Yığıtta



Esnek, ama çekirdek kullanıcı belleğinden kopyalamalıdır. Örnek: x86 üzerinde 32 bit FreeBSD.

Linux 1. ve 2. yolu birleştirir: altı parametreye kadar yazmaçlarda; daha büyük veri (bir tampon, bir yapı) bir yazmaçta adres olarak aktarılır.

Somut Örnek: x86-64 Linux Yazmaç Kuralı

Yazmaç	Sistem çağrısındaki rolü	write(1, msg, 26) için
rax	sistem çağrısı numarası (giriş) · dönüş değeri (çıkış)	1 → 26 (yazılan bayt)
rdi	1. parametre	1 (dosya tanımlayıcısı: standart çıktı)
rsi	2. parametre	msg'nin adresi
rdx	3. parametre	26 (bayt sayısı)
r10, r8, r9	4., 5., 6. parametre	— (write kullanmaz)
rcx, r11	syscall komutu bunların üzerine yazar	işlemci dönüş adresini ve bayrakları buraya kaydeder

−4095 ile −1 arasındaki bir dönüş değeri hata demektir: sarmalayıcı hata numarasını errno'ya yazar ve programınıza −1 döndürür.

UYGULAMA

Örnek 1: Aynı Mesaj, Üç Düzey Derinlikte

```c
int main(void)
{
    printf("1) printf  -> libc -> write\n");     /* düzey 1 */
    fflush(stdout);

    /* düzey 2 */
    if (write(1, "2) write() -> libc wrapper\n", 27) != 27)
        return 1;

    /* düzey 3: sarmalayıcının yaptığını elle yapıyoruz.
     *   rax = 1   write'ın sistem çağrısı numarası (x86-64)
     *   rdi = 1   1. parametre: dosya tanımlayıcısı (stdout)
     *   rsi = msg 2. parametre: baytların adresi
     *   rdx = 26  3. parametre: bayt sayısı
     * 'syscall' komutu çekirdeğe tuzak kurar; çekirdeğin
     * dönüş değeri rax'te geri gelir.                  */
    static const char msg[] = "3) syscall instruction!!!\n";
    long ret;
    __asm__ volatile ("syscall"
                      : "=a"(ret)                /* çıkış: rax */
                      : "a"(1L), "D"(1L), "S"(msg), "d"(26L)
                      : "rcx", "r11", "memory"); /* bozulanlar */
    printf("kernel returned %ld (bytes written)\n", ret);
    return 0;
}
```

Neye dikkat edin?

- 3. düzey, write() sarmalayıcısının yaptığını elle yapar: numara rax'e, parametreler rdi, rsi, rdx'e, sonra syscall.
- strace üç düzeyi birbirinden ayıramaz: çekirdek için hepsi aynıdır.

```
$ strace ./raw_write
write(1, "1) printf  -> libc -> write\n", 28) = 28
write(1, "2) write() -> libc wrapper\n", 27) = 27
write(1, "3) syscall instruction!!!\n", 26) = 26
write(1, "kernel returned 26 (bytes writte"..., 35) = 35
exit_group(0)                           = ?
```

Sistem Çağrılarının Altı Kategorisi

Kategori	Tipik istekler	Linux (POSIX)	Windows API
Süreç denetimi	oluştur, sonlandır, yükle, bekle	fork(), execve(), exit(), wait4()	CreateProcess(), ExitProcess(), WaitForSingleObject()
Dosya yönetimi	oluştur, aç, oku, yaz, kapat	open(), read(), write(), close()	CreateFile(), ReadFile(), WriteFile(), CloseHandle()
Aygıt yönetimi	iste, oku, yaz, yapılandır	ioctl(), read(), write()	SetConsoleMode(), ReadConsole(), WriteConsole()
Bilgi bakımı	saat, tarih, kimlikler, sistem verisi	getpid(), uname(), clock_gettime()	GetCurrentProcessId(), GetSystemTime()
İletişim	boru, paylaşımlı bellek, ağ	pipe(), mmap(), socket()	CreatePipe(), MapViewOfFile()
Koruma	izinler, sahiplik	chmod(), chown(), umask()	SetFileSecurity(), SetSecurityDescriptorGroup()

ÇÖZÜMLÜ ÖRNEK

Bir Dosyayı Kopyalamak Kaç Sistem Çağrısı Gerektirir?

Klasik sıra

- İki dosya adını iste (G/Ç).
- Girdi dosyasını aç; hata varsa bir mesaj yazdır ve çık (dosya yönetimi, süreç denetimi).
- Çıktı dosyasını oluştur.
- Döngü: bir blok oku, bir blok yaz; read() 0 döndürene kadar (dosya sonu).
- İki dosyayı da kapat, başarıyı bildir, çık.
- Her adım bir ya da daha fazla sistem çağrısıdır; programın kendisini yüklemek de öyle.

```
$ strace -c cp in.bin out.bin   # in.bin = 1 MiB
 calls  syscall            (kısaltıldı)
    26  mmap
    13  close
    11  openat
    10  fstat
     9  read
     2  copy_file_range
   102  total
```

102 çağrının yalnızca 2'si veriyi taşır: modern cp, kopyalamayı çekirdeğin içinde yapmasını ister. Çağrıların çoğu programı ve kütüphanelerini yükler.

Sistem Programları: Çoğu Kullanıcının Gördüğü İşletim Sistemi

İşletim sistemiyle birlikte gelen ve sizin adınıza sistem çağrısı yapan programlar. Kullanıcı modunda çalışırlar.

Dosya yönetimi

ls, cp, mv, rm, mkdir; Dosya Gezgini, Finder.

Durum bilgisi

date, df, free, ps, top; Görev Yöneticisi; sistem günlükleri.

Dosya düzenleme

Metin düzenleyiciler (nano, vim, Not Defteri); arama araçları (grep, find).

Programlama desteği

Derleyiciler, çeviriciler, hata ayıklayıcılar, yorumlayıcılar: gcc, gdb, python.

Yükleme ve yürütme

Programları ve kütüphaneleri belleğe yerleştiren yükleyiciler ve bağlayıcılar.

Arka plan hizmetleri

Daemon'lar / hizmetler: sshd, cron, yazdırma kuyruğu. Açılışta başlar.

ARA

10 Dakika Mola

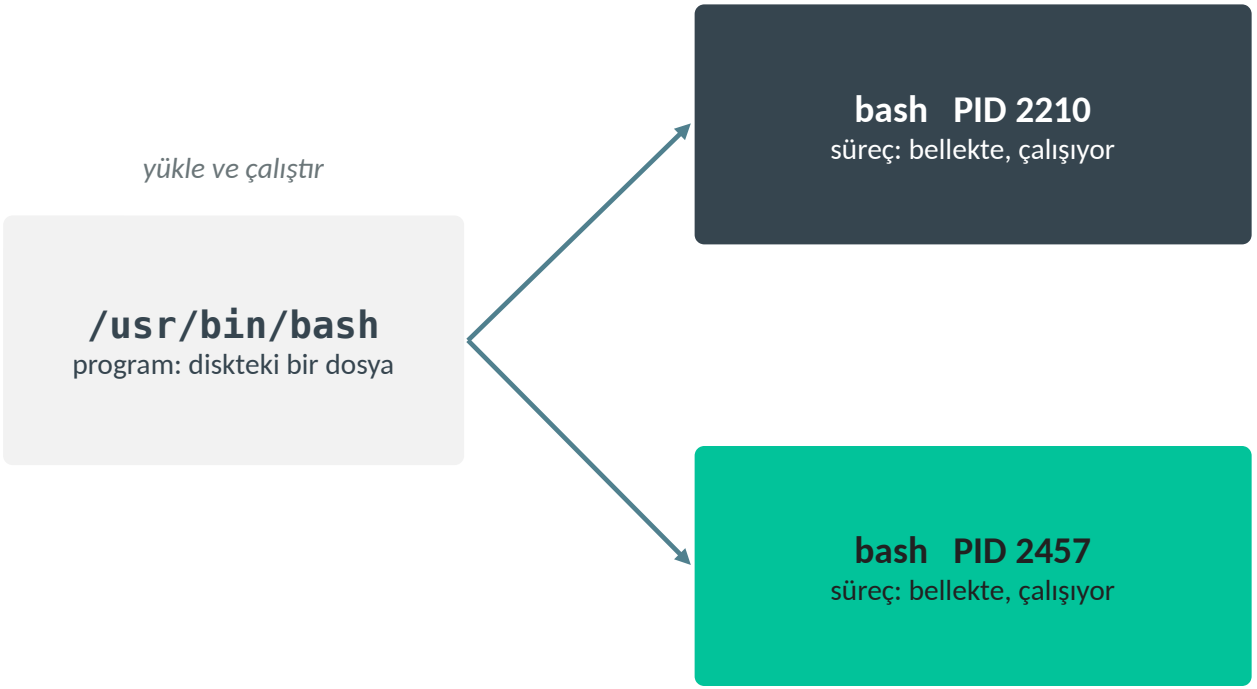
Sırada: süreç — işletim sisteminin yönettiği iş birimi.

BÖLÜM 3

Süreç Kavramı

Yürütülmekte olan bir program.

Program ve Süreç



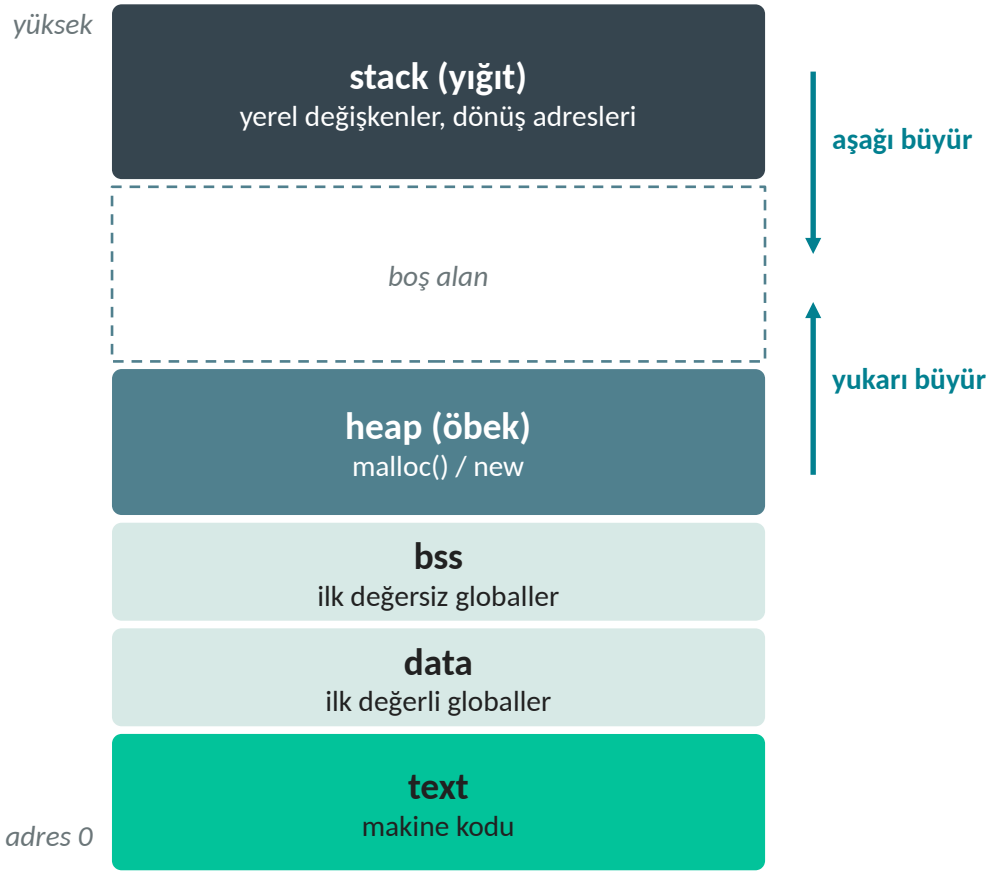
bir program, iki süreç (iki terminal penceresi)

Tanımlar

- **Program:** edilgen; çalıştırılabilir bir dosyada saklanan komutlar.
- **Süreç (process):** etkin; yürütülmekte olan program: kodu artı o anki etkinliği (program sayacı, yazmaçlar), belleği ve açık dosyaları.
- Program belleğe yüklendiğinde sürece dönüşür.

Benzetme: yemek tarifi programdır; onu pişirmek süreçtir. İki aşçı aynı tarifi aynı anda, her biri kendi malzemesi ve kendi ilerlemesiyle uygulayabilir.

Bellekteki Bir Süreç: Beş Bölge



yüklendikten sonra boyutu sabit: text, data, bss · çalışırken büyüyüp küçülür: heap, stack

Ne nerede durur (C'de)

- **text:** main() ve diğer tüm fonksiyonların derlenmiş kodu; salt okunur.
- **data:** int initialised = 42; (değeri olan global)
- **bss:** int uninitialised; (değeri olmayan global; işletim sistemi sıfırla doldurur)
- **heap (öbek):** malloc() ile alınan bellek; free()'ye kadar yaşar.
- **stack (yığıt):** bir fonksiyon içindeki int local = 7; her etkin çağrı için bir çerçeve, fonksiyon dönünce kaldırılır.
- Öbek ve yığıt, aradaki boş alanda birbirine doğru büyür.

UYGULAMA

Örnek 2: Her Bölgenin Adresini Yazdırmak

```c
#include <stdio.h>
#include <stdlib.h>

int initialised = 42;   /* değeri olan global   -> data */
int uninitialised;      /* değeri olmayan global -> bss  */

int main(void)
{
    int local = 7;                  /* yerel değişken -> yığıt */
    int *dynamic = malloc(sizeof *dynamic);     /* malloc -> öbek */

    printf("text  main()        %p\n", (void *)main);
    printf("data  initialised   %p\n", (void *)&initialised);
    printf("bss   uninitialised %p\n", (void *)&uninitialised);
    printf("heap  *dynamic      %p\n", (void *)dynamic);
    printf("stack local         %p\n", (void *)&local);

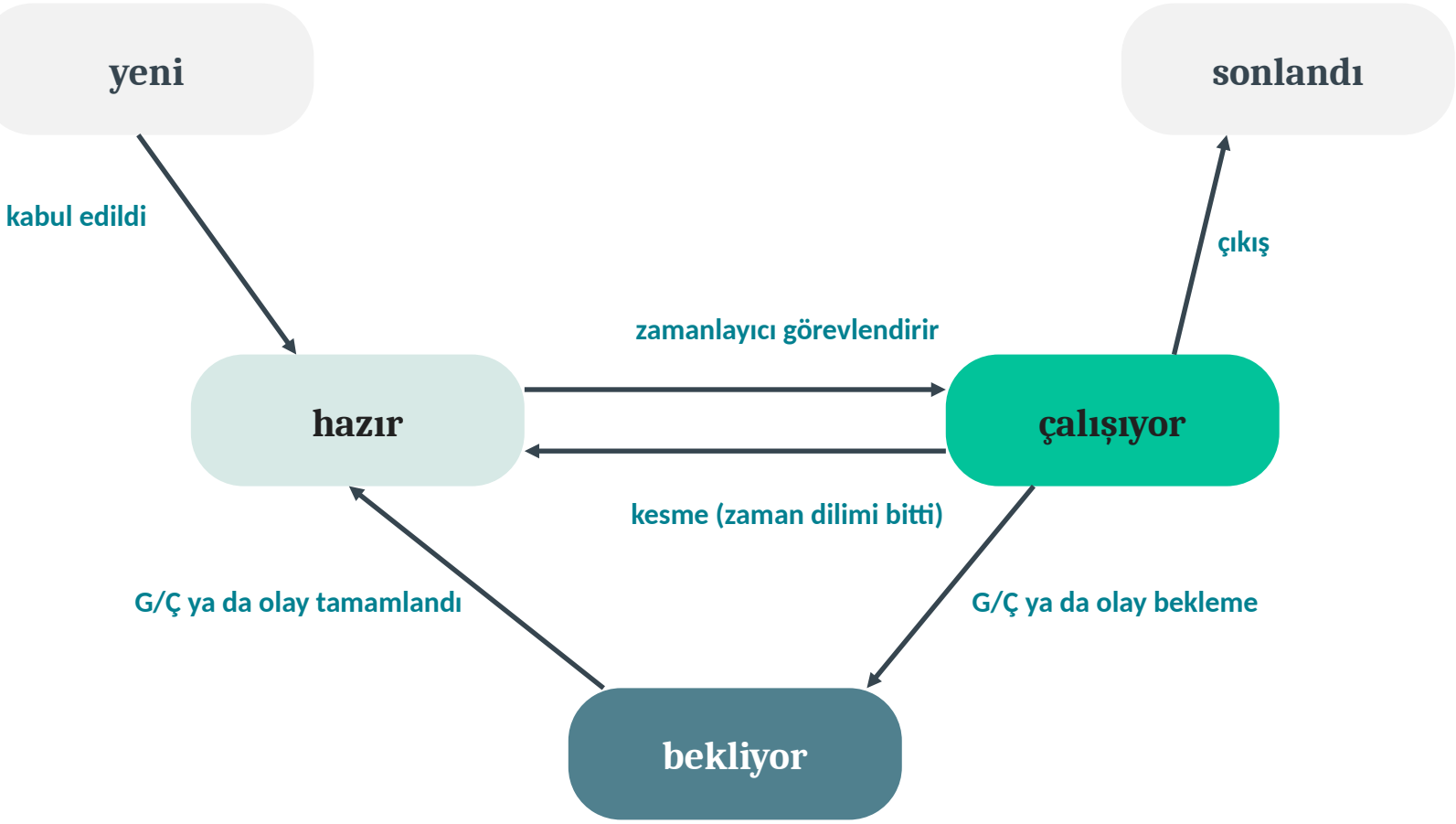
    free(dynamic);
    return 0;
}
```

Neye dikkat edin?

- Sıra: text < data < bss < heap; yığıt çok daha yukarıda.
- İki kez çalıştırın: her adres değişir. Linux saldırıları zorlaştırmak için bölgeleri rastgele yerleştirir (ASLR).

```
$ ./mem_layout
text  main()        0x555d9ba160c0
data  initialised   0x555d9ba19010
bss   uninitialised 0x555d9ba19018
heap  *dynamic      0x555db649d2a0
stack local         0x7ffe2b4f6ab4
```

Beş Durumlu Süreç Modeli



Kurallar

- İşlemci çekirdeği başına bir çalışan süreç.
- Aynı anda çok sayıda süreç hazır ya da bekliyor olabilir.
- Hazır ve bekleyen süreçler kuyruklarda durur: PCB'lerinin bağlı listeleri.
- Linux ps'te: R = çalışıyor ya da hazır, S = bekliyor (uyuyor), Z = zombi.

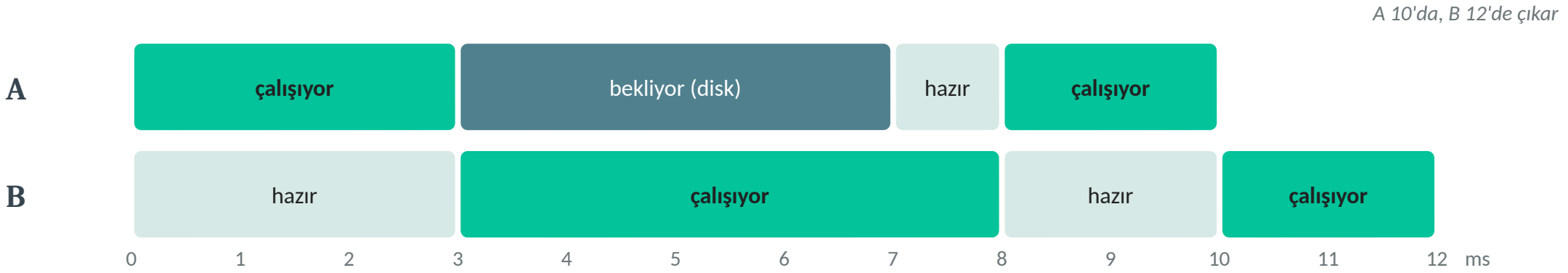
Her Geçişi Ne Tetikler?

Geçiş	Tetikleyen	Gündelik örnek
yeni → hazır	işletim sistemi PCB'yi oluşturdu ve programı yükledi	./mem_layout yazıp Enter'a basarsınız
hazır → çalışıyor	zamanlayıcı süreci görevlendirir (dispatch)	işlemci çekirdeği boşalır ve zamanlayıcı onu seçer
çalışıyor → hazır	bir kesme, genellikle zamanlayıcı saat (zaman dilimi doldu)	başkaları çalışabilsin diye uzun bir hesap duraklatılır
çalışıyor → bekliyor	beklemesi gereken bir sistem çağrısı (G/Ç, çocuk, sleep)	diskten read(); bir çocuk için wait()
bekliyor → hazır	beklenen olay gerçekleşir (çoğunlukla bir kesme)	disk kesmesi verinin geldiğini bildirir
çalışıyor → sonlandı	exit() ya da ölümcül bir hata	main() döner; program sıfıra böler

Bekliyor → çalışıyor geçişi olanaksızdır: uyanan süreç önce hazır kuyruğuna katılır. Sırada kimin çalışacağını seçmek işlemci zamanlamasıdır (4–5. haftalar).

ÇÖZÜMLÜ ÖRNEK

İki Süreç, Tek İşlemci: Zaman İçinde Durumlar



Olaylar

- **t = 3:** A read() çağırır → bekliyor; B görevlendirilir.
- **t = 7:** disk kesmesi → A hazır olur (işlemci B'de kalır).
- **t = 8:** zamanlayıcı kesmesi B'nin 5 ms'lik dilimini bitirir → A çalışır.
- **t = 10:** A çıkar → B 12'de çıkana kadar çalışır.

```
Gereken işlemci işi: A = 3+2 = 5 ms
                     B = 5+2 = 7 ms

Art arda (örtüşme yok):
  A 9 ms (işlemci 4 boşta) + B 7 ms = 16 ms
  İşlemci meşgul 12/16 = %75

İç içe (yukarıdaki gibi):
  12 ms, işlemci meşgul 12/12 = %100
```

SÜREÇ DENETİM BLOĞU

PCB: İşletim Sisteminin Bir Süreç Hakkında Bildiği Her Şey

süreç durumu

süreç kimliği (PID), ebeveyn PID

program sayacı

işlemci yazmaçları

işlemci zamanlama bilgisi

bellek yönetimi bilgisi

hesap bilgisi

G/Ç durumu: açık dosyalar, aygıtlar

her süreç için bir PCB, çekirdek belleğinde tutulur

Alanlar ne işe yarar?

- **Durum:** yeni, hazır, çalışıyor, bekliyor ya da sonlandı.
- **Program sayacı ve yazmaçlar:** süreç yeniden çalıştığında nereden devam edileceği ve geri yüklenecek değerler.
- **Zamanlama bilgisi:** öncelik, zamanlama kuyruklarına işaretçiler.
- **Bellek bilgisi:** sürecin bölgelerinin nerede olduğu (sayfa tabloları, sınırlar; 6–7. haftalar).
- **Hesap bilgisi:** kullanılan işlemci zamanı, zaman sınırları, süreç numaraları.
- **G/Ç durumu:** açık dosyaların ve ayrılmış aygıtların listesi.
- Linux'ta PCB, yüzlerce alanı olan bir C yapısıdır: struct task_struct.

Bir PCB'ye Göz Atmak: /proc/<pid>/status

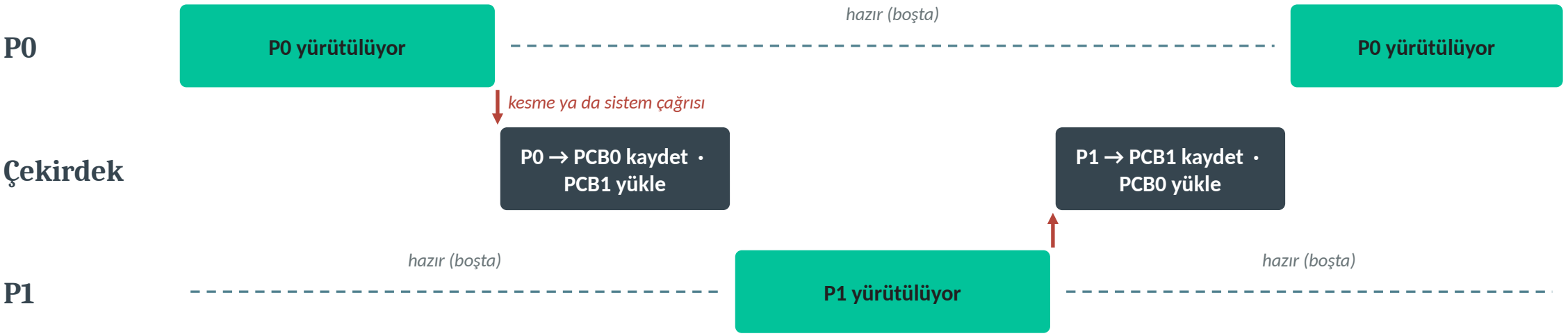
Linux, her sürecin PCB'sinin bazı kısımlarını /proc dizininde bir metin dosyası olarak gösterir.

```
$ sleep 30 &
$ grep -E "^(Name|State|Pid|PPid|Uid|VmSize|VmRSS|
  Threads|voluntary)" /proc/$!/status
Name:   sleep
State:  S (sleeping)
Pid:    27271
PPid:   27265
Uid:    0       0       0       0
VmSize:     2724 kB
VmRSS:      1696 kB
Threads:        1
voluntary_ctxt_switches:        2
nonvoluntary_ctxt_switches:     0
```

alan → PCB girdisi

`State`	→	**süreç durumu**
`Pid, PPid`	→	**kimlik**
`Uid`	→	**sahip (koruma)**
`VmSize, VmRSS`	→	**bellek yönetimi bilgisi**
`Threads`	→	**4. hafta!**
`…ctxt_switches`	→	**hesap bilgisi**

Bağlam Değişimi: İşlemciyi Bir Süreçten Diğerine Geçirmek



Ne kaydedilir ve geri yüklenir?

Program sayacı, işlemci yazmaçları, yığıt işaretçisi, süreç durumu ve bellek yönetimi bilgisi (hangi sayfa tablosunun etkin olduğu). Hepsi PCB'de durur.

Koyu kutular süresince hiçbir süreç yararlı iş yapmaz: bağlam değişimi saf ek yüktür.

ÇÖZÜMLÜ ÖRNEK

Bağlam Değişimi Ne Kadara Mal Olur?

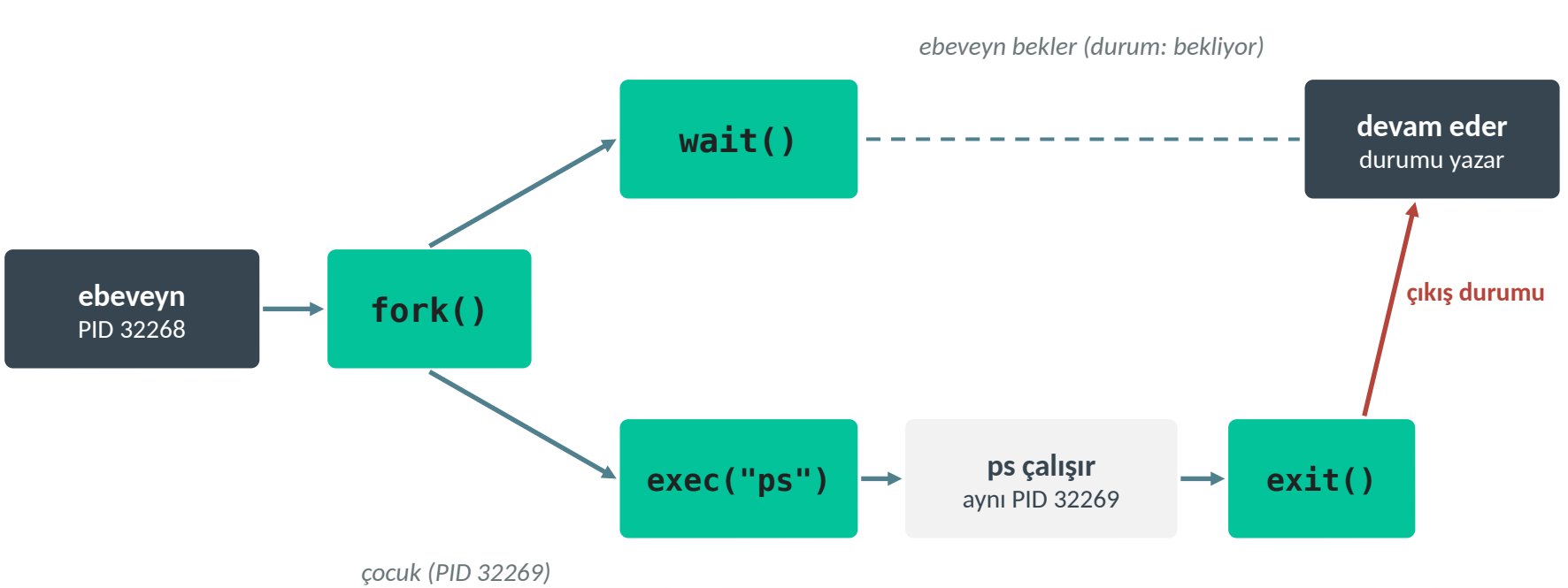
Model

- Her süreç q zaman dilimi kadar çalışır, sonra işletim sisteminin geçiş için s süresine ihtiyacı olur.
- Kaybedilen işlemci zamanı oranı:
- ek yük = s / (q + s)
- s = 5 µs varsayalım (temsilî; gerçek geçişler birkaç µs sürer, önbellekler yeniden dolarken daha fazla).
- Örnek: saniyede 1000 geçiş × 5 µs = saniyede 5 ms = %0,5.

```
zaman dilimi q   ek yük s/(q+s)
100   ms         %0.005
 10   ms         %0.05
  1   ms         %0.50
  0.1 ms         %4.76
  0.01 ms        %33.3
  0.005 ms       %50     (q = s)
```

Kısa dilimler hızlı tepki verir ama işlemciyi harcar; uzun dilimler az harcar ama hantal hissettirir. q'yu seçmek bir Round-Robin sorusudur (5. hafta).

Süreç Oluşturmak: fork(), exec(), wait()



Windows iki adımı tek çağrıda yapar: CreateProcess().

fork()

Çağıran süreci kopyalar. İki kez döner: çocukta 0, ebeveynde çocuğun PID'si (hatada −1).

exec()

Süreçteki programı yenisiyle değiştirir. PID aynı kalır; başarılı olursa asla geri dönmez.

wait()

Ebeveyn bir çocuk bitene kadar uyur, sonra çıkış durumunu toplar.

UYGULAMA

Örnek 3: C ile fork, exec ve wait

```c
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);            /* fork'tan ÖNCE tamponu boşalt */

    pid_t pid = fork();        /* biri çağırır, ikisi döner */

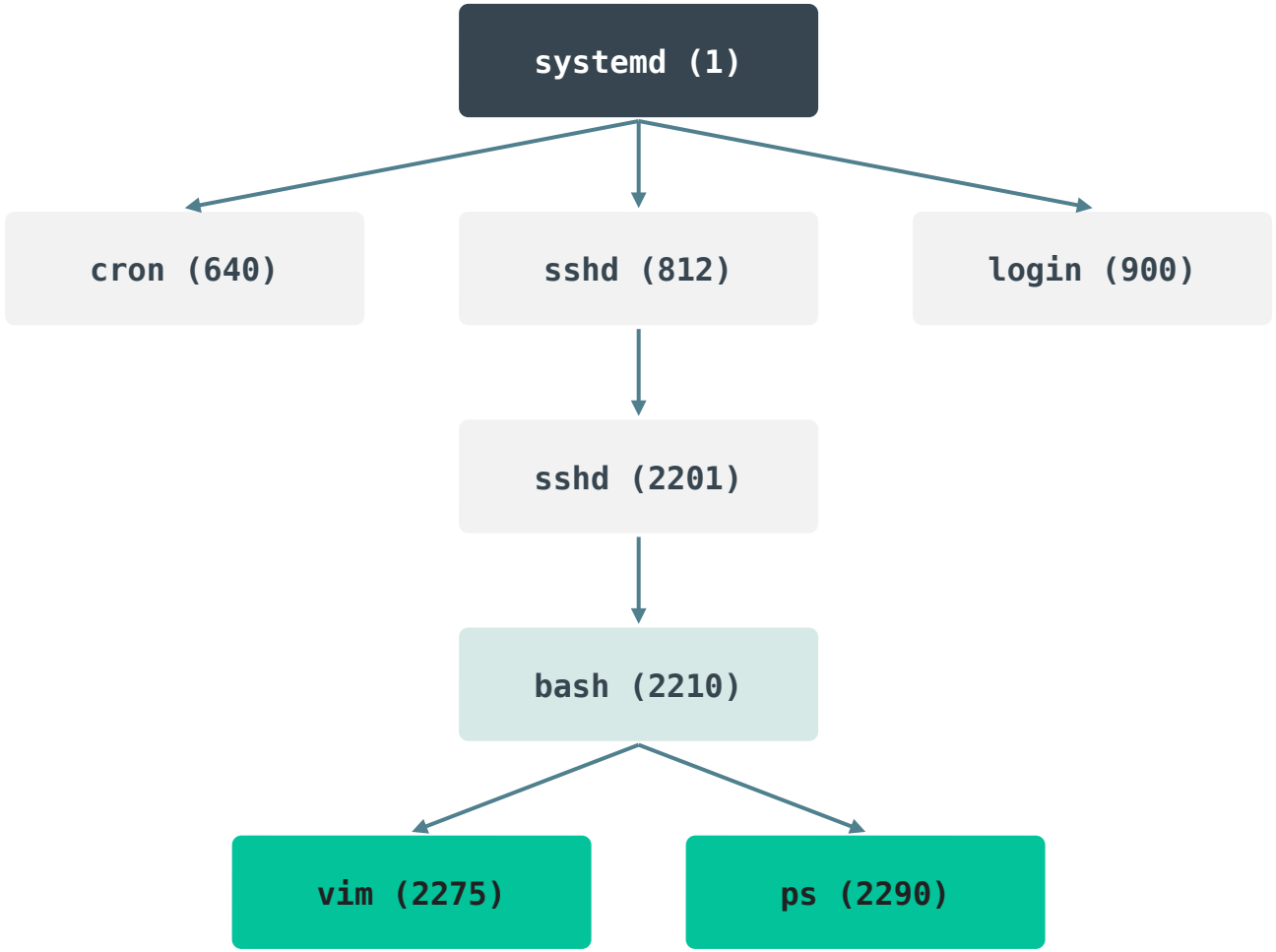
    if (pid == 0) {            /* ---- çocuk: fork 0 döndürdü */
        printf("child : PID %d, my parent is %d\n",
               getpid(), getppid());
        fflush(stdout);
        char list[32];         /* ps -p için "ebeveyn,çocuk" */
        snprintf(list, sizeof list, "%d,%d", getppid(), getpid());
        execlp("ps", "ps", "-o", "pid,ppid,stat,comm",
               "-p", list, NULL);   /* bu programın yerine geçer */
        exit(127);             /* yalnızca exec başarısızsa */
    }
    int status;                /* ---- ebeveyn */
    waitpid(pid, &status, 0);  /* çocuk bitene kadar uyu */
    printf("parent: child %d finished, exit status %d\n",
           pid, WEXITSTATUS(status));
}
/* hata denetimli tam program: code/fork_exec_wait.c */
```

Neye dikkat edin?

- Aynı if testi, iki farklı yanıt: çocuk 0 görür, ebeveyn çocuğun PID'sini.
- ps, PID 32269'u "ps" olarak listeler: exec PID'yi korudu.

```
$ ./fork_exec_wait
parent: my PID is 32268
child : PID 32269, my parent is 32268
  PID  PPID STAT COMMAND
32268 32251 S    fork_exec_wait
32269 32268 R    ps
parent: child 32269 finished,
        exit status 0
```

Süreç Ağacı ve Süreç Kimlikleri



tipik bir Linux sunucusu (PID'ler temsilî)

```
$ bash -c 'sleep 5 & sleep 5 & pstree -p $$'
bash(6297)-+-pstree(6300)
           |-sleep(6298)
           `-sleep(6299)
```

Süreç kimlikleri

- Her sürecin benzersiz bir PID'si vardır ve ebeveyninin PID'sini (PPID) bilir.
- PID 1'i (systemd ya da init) çekirdek açılışta başlatır: tüm kullanıcı süreçlerinin atası.
- PID'ler süreç bitince yeniden kullanılır; üst sınır /proc/sys/kernel/pid_max'tedir.

Zombiler ve Yetimler

Bir süreç bittiğinde, PCB girdisi biri çıkış durumunu toplayana kadar yaşamaya devam etmelidir.

Zombi (zombie)

- Çocuk sonlandı, ama ebeveyni henüz wait() çağırmadı.
- Bellek serbest bırakıldı; yalnızca PCB girdisi (PID, çıkış durumu) kalır.
- Çok sayıda zombi tüm PID'leri tüketebilir.

Yetim (orphan)

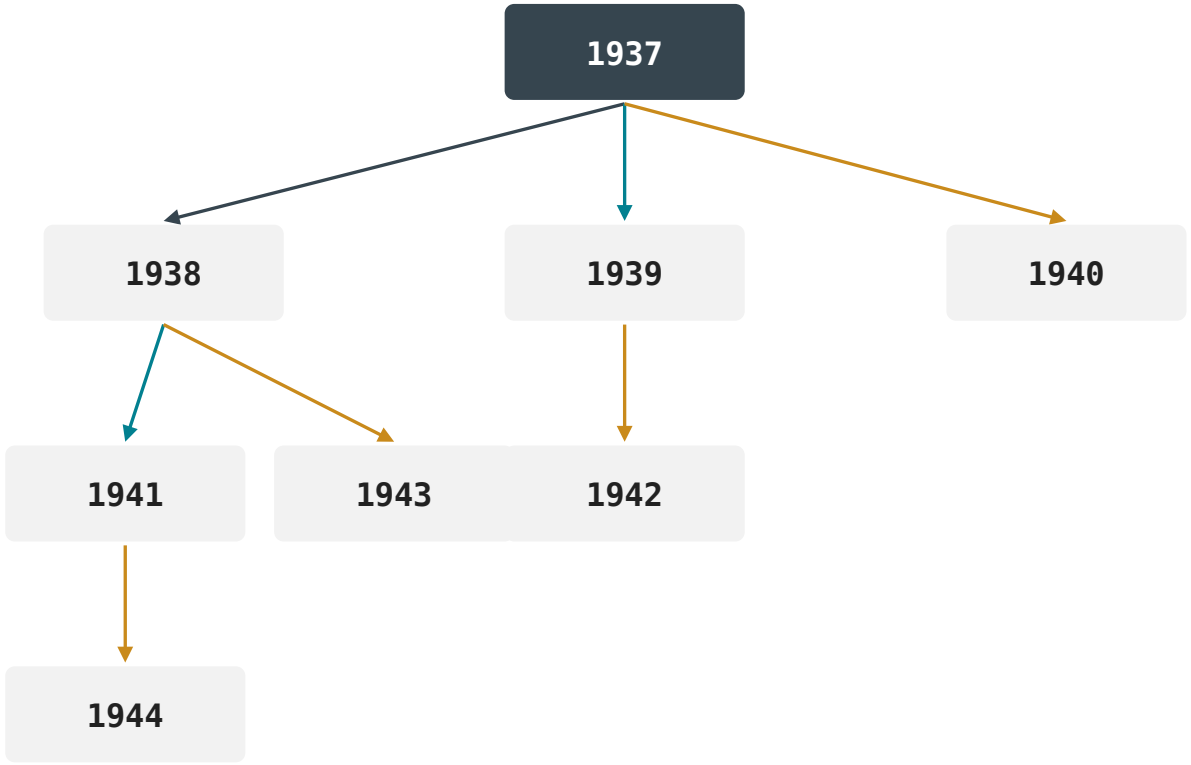
- Çocuk hâlâ çalışırken ebeveyn önce sonlandı.
- Linux onu PID 1'e (ya da bir "subreaper"a) bağlar; o da onun için wait() çağırır.
- Bazı sistemler bunun yerine tüm çocukları öldürür (zincirleme sonlandırma).

```
$ sh -c 'sleep 1 & exec sleep 4' &      # 2 s sonra:
$ ps -o pid,ppid,stat,cmd -p 6385 --ppid 6385
  PID  PPID STAT CMD
 6385  6384 S    sleep 4
 6387  6385 Z    [sleep] <defunct>
```

sleep 4 hiç wait() çağırmaz; bu yüzden biten çocuğu zombi olarak kalır (durum Z).

ALIŞTIRMA

n Kez fork() Çağrısından Sonra Kaç Süreç Olur?



ok rengi = çocuğu hangi fork yarattı: ***1.*** ***2.*** ***3.***

./fork_count 3'ten gerçek PID'ler

```
for (int i = 0; i < n; i++)
    fork();     /* herkes fork eder */
printf("PID %6d  parent %6d\n",
       getpid(), getppid());
```

```
$ for n in 1 2 3 4 5 10; do
>   ./fork_count $n | wc -l; done
2  4  8  16  32  1024      (süreç başına bir satır)
```

Her fork'ta süreç sayısı ikiye katlanır: n fork'tan sonra toplam 2^n süreç olur, bunların $2^n - 1$'i yeni yaratılmıştır. printf 2^n kez çalışır.

KENDİNİZ DENEYİN

Kendi Makinenizdeki Süreçlere Bakın

```
# Linux (laboratuvar, WSL ya da sanal makine)
$ ps -ef | head            # tüm süreçler, PPID ile
$ ps -o pid,ppid,stat,cmd  # süreçleriniz + durum
$ pstree -p | less         # süreç ağacı
$ cat /proc/$$/status      # kabuğunuzun PCB'si
$ strace -c ls             # türe göre sistem çağrıları
$ make && ./fork_count 4   # 3. hafta kod klasörü

# Windows
> tasklist                 # süreçleri listele
# Görev Yöneticisi > Ayrıntılar: PID sütunu
```

Yanıtlanacak sorular

- Kabuğunuzun PID'si nedir ve ebeveyni kimdir?
- Şu anda R durumunda kaç süreç var? Neden bu kadar az?
- Kabuğunuz şimdiye kadar kaç gönüllü bağlam değişimi yaptı?
- ls'in en çok altı sistem çağrısı kategorisinden hangisini kullandığını bulun.

KAPANIŞ

3. Hafta: Temel Çıkarımlar

- **Hizmetler:** kullanıcıya dönük (arayüz, yürütme, G/Ç, dosyalar, iletişim, hatalar) ve sisteme dönük (tahsis, hesap tutma, koruma).
- **Sistem çağrıları:** bir API ve bir libc sarmalayıcısı üzerinden ulaşılır; sistem çağrısı tablosunda bir numarayla seçilir; bir tuzakla girilir.
- **Parametreler:** yazmaçlar, bir bellek bloğu ya da yığıt; x86-64 Linux rax + en çok altı yazmaç kullanır.
- **Süreç:** text, data, bss, heap ve stack bölgeleri olan, yürütülmekte olan program; PCB'siyle tanımlanır.
- **Durumlar:** yeni, hazır, çalışıyor, bekliyor, sonlandı; geçişlere kesmeler ve sistem çağrıları yol açar.
- **Bağlam değişimi ve oluşturma:** geçiş saf ek yüktür, s/(q+s); fork–exec–wait süreç ağacını kurar.

KISA SINAV

Kendinizi Sınayın

1. printf() bir sistem çağrısı mıdır? printf() ile ekran arasında ne olduğunu açıklayın.

2. İşletim sistemine parametre aktarmanın üç yolunu sayın. x86-64 Linux write() için hangisini kullanır?

3. Bir süreç neden bekliyor durumundan doğrudan çalışıyor durumuna geçemez?

4. Bir bağlam değişimi 10 µs, zaman dilimi 1 ms sürüyor. İşlemci zamanının yüzde kaçı ek yüktür?

GELECEK HAFTA

İş Parçacıkları ve İşlemci Zamanlamasının Temelleri

4. haftanın konuları:

- İş parçacıkları (threads): tek süreç, birden çok yürütme akışı
- Kullanıcı iş parçacıkları, çekirdek iş parçacıkları ve çoklu iş parçacığı modelleri
- İşlemci zamanlaması: işlemci–G/Ç patlama döngüsü ve dağıtıcı (dispatcher)
- Zamanlama ölçütleri: işlemci kullanımı, iş hacmi, tamamlanma, bekleme ve yanıt süresi

SORULAR?

Tartışma

Dersten sonra sorularınız olursa dersin Telegram kanalında sorun — yanıttan herkes yararlanır.

Dr. Öğr. Üyesi Hasan Oğuz